4. Analízis modell kidolgozása

48 – GonoszOnosz

Konzulens:

Marcsingó Ádám

Csapattagok

Kálmán Patrik	XG5YQ1	kalmanpatrik1214@gmail.com
Papp Bálint	HND2AH	xypapbal@gmail.com
dr. Grób László Mihály	EAS7U0	laszlo.m.grob@gmail.com
Héja Gergő Zoltán	FEFSRB	hejagergo@gmail.com
Dobronyi Benedek Márk	IMQ1AR	benedek.dobronyi@gmail.com

2026.03.12

3. Analízis modell kidolgozása

3.1 Objektum katalógus

Az objektum katalógusban magyar névvel tünettük fel az objektumokat, viszont az osztályok tervezésénél már angolul készítettük a terveket, az objektum katalógusban feltüntettük az objektum angol nevét is.

3.1.1 Takarító (Játékos)

A takarító játékos célja, hogy a közlekedés minél zökkenőmentesebb legyen. Az ő felelőssége a hókotró flottájának irányítása: eldönti, melyik munkagép merre menjen, milyen fejet használjon. A megtisztított utakért kapott fizetségből ő vásárol új eszközöket, alapanyagokat vagy akár új hókotrókat a város boltjában.
Angolul: Cleaner(Player)

3.1.2 Buszvezető (Játékos)

A buszvezető feladata, hogy a rábízott busszal a lehető legtöbb fordulót tegye meg a kijelölt végállomások között az adott műszak alatt. Felelőssége, hogy folyamatosan figyelje az útviszonyokat és okosan navigáljon, elkerülve a mély havat, a csúszós jégpáncélokat és a balesetek miatt kialakult dugókat.
Angolul: BusDriver(Player)

3.1.3 Hókotró (Munkagép)

A takarító játékos által irányított gépezet, amely a város útjait járja. Felelőssége a rászerelt tisztítófej segítségével a hó és a jég eltakarítása az utakról – egyszerre mindig csak egy sávon. A munkájáért pénzt termel a takarítónak. Ha elfogy az üzemanyaga (só vagy biokerozin), a felszerelt fej hatástalanná válik. Hóláncai miatt nem csúszik jégen.
Angolul:SnowPlow

3.1.4 Busz

A városban közlekedő, buszvezető által irányított tömegközlekedési jármű, kizárólag a két végállomása között ingázik. Ha jégre fut és megcsúszik, vagy egy másik járműbe rohan, egy időre mozgásképtelenné válik. Letapossa a havat, hozzájárulva a jégpáncél kialakulásához.
Angolul:Bus

3.1.5 Autó

A városlakók automatizált járműve, amely önállóan próbál eljutni a lakástól a munkahelyig a legrövidebb úton. Ha vékony hórétegen halad át, azt letapossa – amiből idővel jégpáncél lesz. Ha túl mély hóba ér, elakad; ha jégen megcsúszik, balesetet okoz és elzárja a sávot. Szomszédos szabad sávba tud átsorolni.
Angolul:Car

3.1.6 Söprő fej

A hókotró alapfelszerelése. Feladata az úton lévő havat oldalra tolni – a szomszédos sávba vagy az út mellé. A jéggel és a feltört jéggel szemben teljesen tehetetlen. Üzemanyagot nem igényel.
Angolul: SweepHead

3.1.7 Hányó fej

A söprő fejhez hasonló elven működik, de a havat jó messzire szórja el, az úttesttől távolra (jellemzően a járdára). A lefagyott jeget ez sem tudja felszedni. Üzemanyagot nem igényel.
Angolul:ThrowerHead

3.1.8 Jégtörő fej

Speciális eszköz, amelynek egyetlen feladata a veszélyes jégpáncél zúzása. Havat nem tud takarítani. A feltört jeget hó formájában az úton hagyja, ezzel a sáv havassá válik, de a csúszásveszély megszűnik. Üzemanyagot nem igényel.
Angolul:IceBreakerHead

3.1.9 Sószóró fej

Sót juttat az útfelületre. A kiszórt só idővel elolvasztja a meglévő havat és jeget, emellett egy ideig megakadályozza, hogy az újonnan hulló hó megmaradjon. Működéséhez folyamatos só-utánpótlás kell; ha a készlet elfogy, a fej hatástalanná válik.
Angolul:SaltHead

3.1.10 Sárkány fej

A legdrágább és leghatékonyabb eszköz: egy gázturbina, amely azonnal, nyomtalanul elolvast mindent a hókotró előtt – havat és jeget egyaránt. Üzemeltetéséhez biokerozinra van szükség. Nem akadályozza meg az újabb hó lerakódását.
Angolul: DragonHead

3.1.11 Sáv

Az úthálózat gráfjának alapcsomópontja: egy forgalmi sáv, amelyen a járművek közlekednek. Maga is Field, azaz szomszédait (más sávokat, kereszteződéseket) asszociáción keresztül ismeri. Felelőssége nyilvántartani a rajta lévő hóréteg vastagságát, a jégpáncél állapotát és a rajta tartózkodó járműveket.
Angolul:Lane

3.1.12 Telephely

A hókotrók bázisa és egyben Field a gráfban. Felelőssége a hókotrók fogadása és karbantartásának koordinálása: fejcsere, só- és biokerozin-utántöltés. A beépített áruháznak is helyet ad.
Angolul:HomeBase

3.1.13 Beépített áruház

A telephelyen található bolt, ahol a takarító játékos el tudja költeni az úttisztításért kapott pénzét. Felelőssége az eladható termékek (új hókotrók, fejek, só, biokerozin) nyilvántartása és értékesítése.
Angolul:IntegratedMarket

3.1.14 Végállomás

A buszjáratok végpontja, egyben Field a gráfban. Felelőssége a fordulók regisztrálása: amikor egy busz megérkezik, jelzi a buszvezetőnek a forduló teljesítését.
Angolul:Terminal

3.1.15 Épület (Lakás/Munkahely)

Az autók által felkeresett általános épület, egyben Field a gráfban. Az autók a lakásuk és munkahelyük között ingáznak. Felelőssége, hogy parkolóhelyet biztosítson az autóknak (így az útsávot szabadon hagyják), és navigációs célpontként szolgáljon.
Angolul:GeneralBuilding

3.1.16 Só

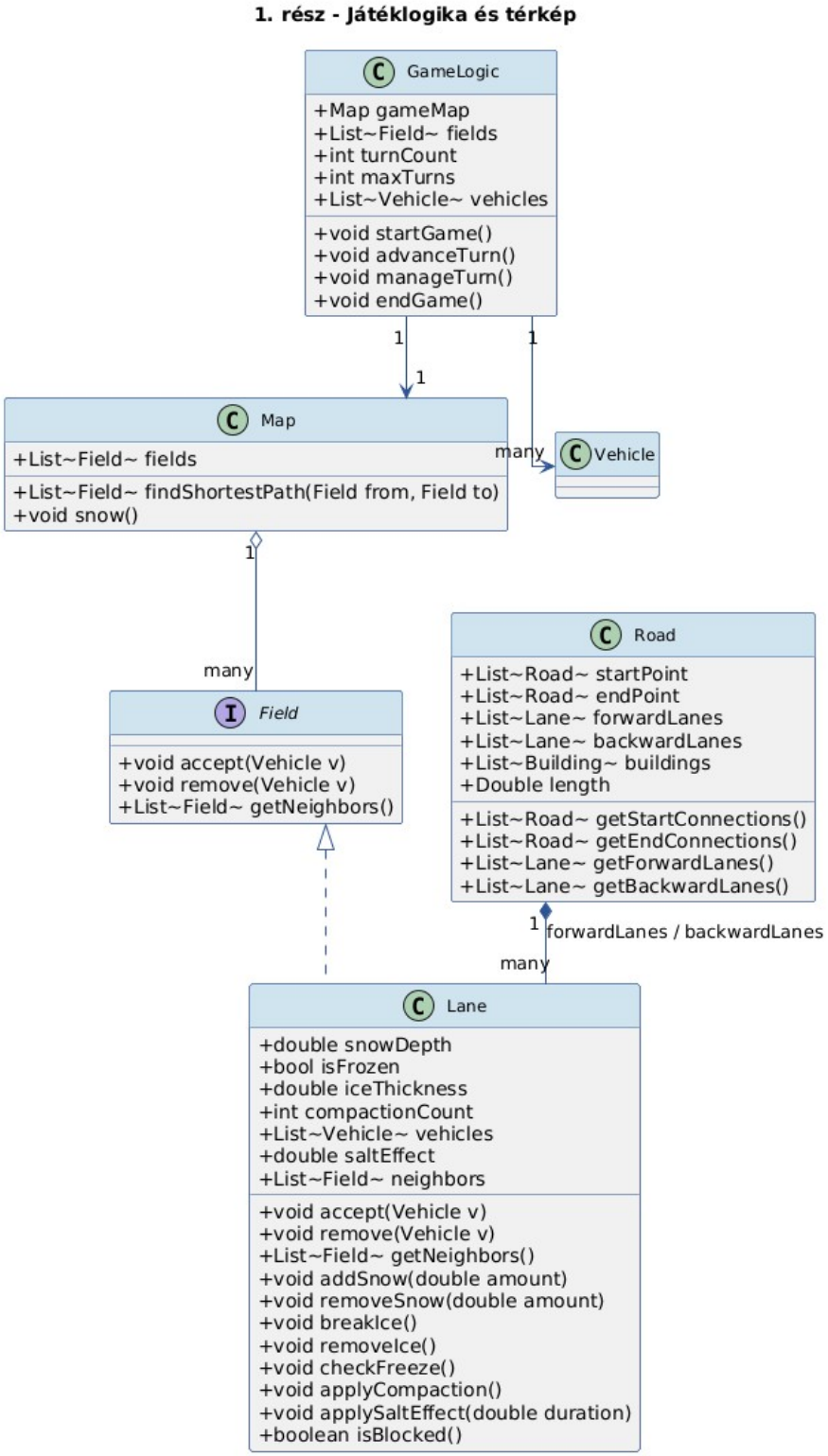
A sószóró fej működéséhez szükséges fogyóeszköz. Felelőssége, hogy a sószóró fejnek rendelkezésre álljon, és a tisztítás során fogyjon. A boltban vásárolható, a telephelyen töltendő be.
Angolul:Salt

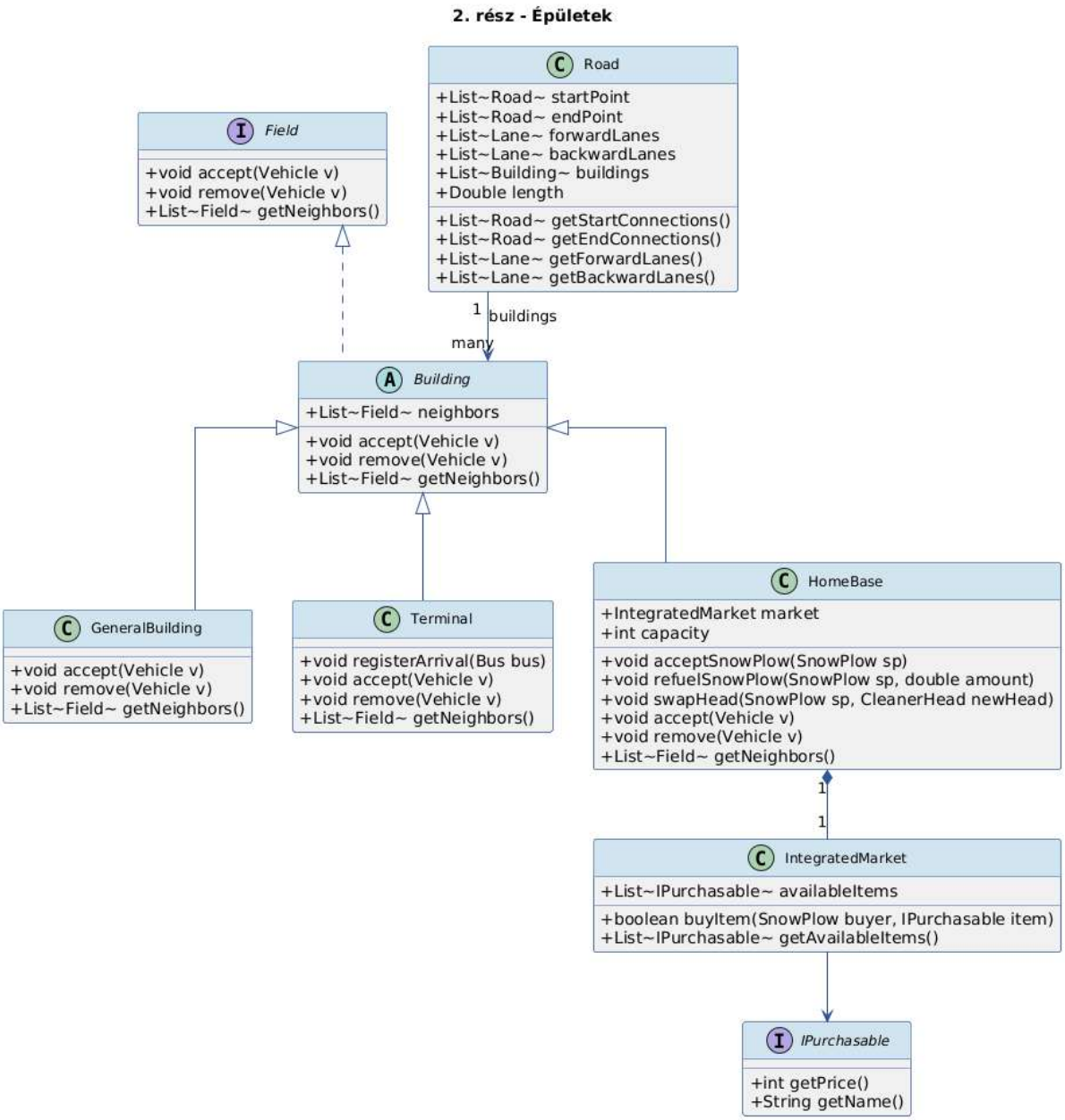
3.1.17 Biokerozin

A sárkány fej működéséhez szükséges fogyóeszköz. Felelőssége, hogy a sárkány fejnek rendelkezésre álljon, és a tisztítás során fogyjon. A boltban vásárolható, a telephelyen töltendő be.Statikus struktúra diagramok
Angolul:Biokerozine

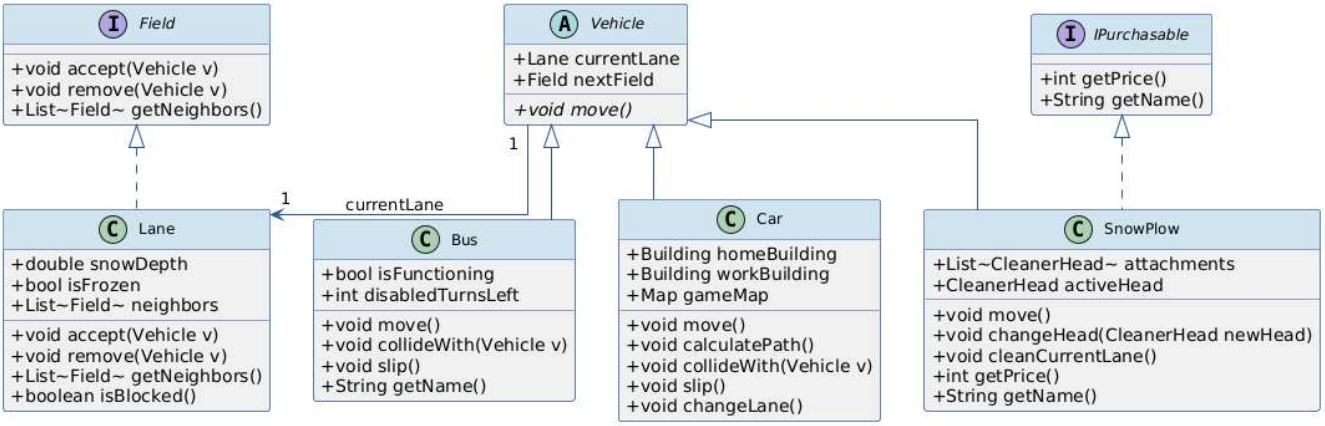
3.2 Statikus struktúra diagramok

Az alábbi osztálydiagram a játék analízis modelljének statikus struktúráját mutatja be. A pályaelemek (sávok, kereszteződések, épületek) mind a Field interfész megvalósításai, így az úthálózat koordináta-független, gráfalapú modellként jelenik meg.

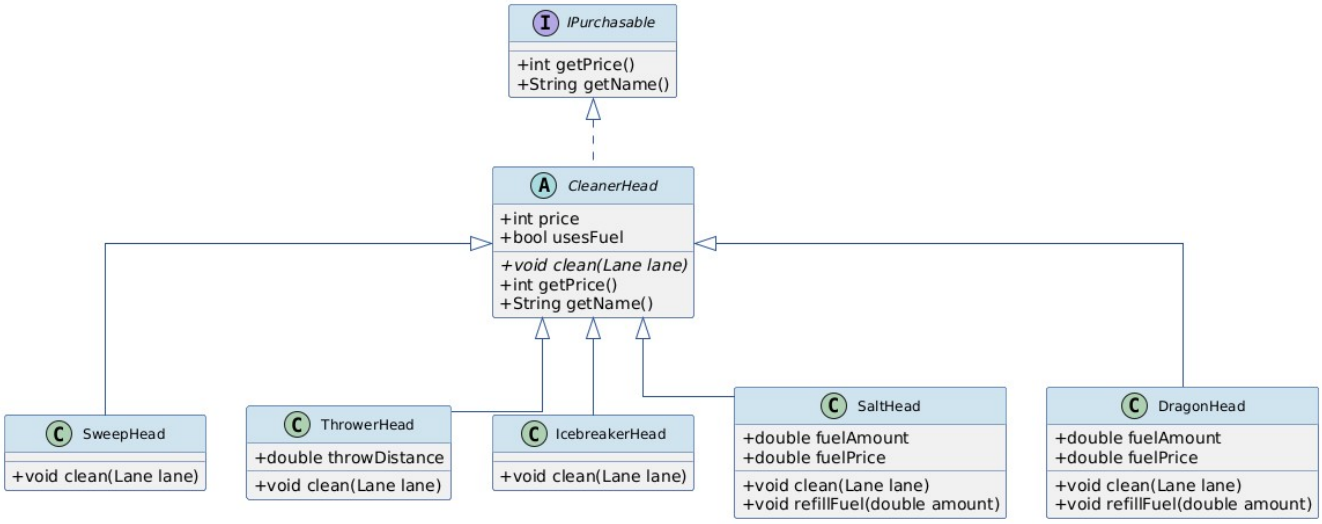



2. rész - Épületek
Road
+List~Road~ startPoint
+List~Road~ endPoint
+List~Lane~ forwardLanes
+List~Lane~ backwardLanes
+List~Building~ buildings
+Double length
+List~Road~ getStartConnections()
+List~Road~ getEndConnections()
+List~Lane~ getForwardLanes()
+List~Lane~ getBackwardLanes()
Field
+void accept(Vehicle v)
+void remove(Vehicle v)
+List~Field~ getNeighbors()
1
buildings
many
Building
+List~Field~ neighbors
+void accept(Vehicle v)
+void remove(Vehicle v)
+List~Field~ getNeighbors()
GeneralBuilding
+void accept(Vehicle v)
+void remove(Vehicle v)
+List~Field~ getNeighbors()
Terminal
+void registerArrival(Bus bus)
+void accept(Vehicle v)
+void remove(Vehicle v)
+List~Field~ getNeighbors()
HomeBase
+IntegratedMarket market
+int capacity
+void acceptSnowPlow(SnowPlow sp)
+void refuelSnowPlow(SnowPlow sp, double amount)
+void swapHead(SnowPlow sp, CleanerHead newHead)
+void accept(Vehicle v)
+void remove(Vehicle v)
+List~Field~ getNeighbors()
1
1
IntegratedMarket
+List~IPurchasable~ availableItems
+boolean buyItem(SnowPlow buyer, IPurchasable item)
+List~IPurchasable~ getAvailableItems()
IPurchasable
+int getPrice()
+String getName()

3. rész - Járművek



4. rész - Hókotró fejek



3.3 Osztályok leírása

Az osztályok ABC sorrendben következnek. Interfészek esetén az Interfészek és Attribútumok pont elmarad.

3.3.1 Building <<abstract>> (Épület)

Felelősség

Az összes épülettípus (GeneralBuilding, HomeBase, Terminal) közös absztrakt ősosztálya. Implementálja a Field interfészt. Felelőssége, hogy egységes alapot biztosítson az épületeknek a szomszédossági kapcsolatok kezeléséhez.

Ősosztályok

Nincs

Interfészek

Field

Attribútumok

- List<Field> neighbors: a szomszédos mezők listája (Field interfészből)

Metódusok

- void accept(Vehicle v): fogadja az érkező járművet (Field-ből felüldefiniálva)
- void remove(Vehicle v): eltávolítja a távozó járművet (Field-ből felüldefiniálva)
- List<Field> getNeighbors(): visszaadja a szomszédos mezőket (Field-ből)

3.3.2 Bus (Busz)

Felelősség

A buszvezető játékos által irányított jármű, amely két végállomás között közlekedik. Fordulókat teljesít; ütközés vagy csúszás esetén meghatározott ideig mozgásképtelenné válik és elzárja a sávot.

Ősosztályok

Vehicle

Interfészek

Nincs

Attribútumok

- bool isFunctioning: hamis értéke esetén a busz ütközés miatt mozgásképtelen
- int disabledTurnsLeft: hány körig marad még mozgásképtelen

Metódusok

- void move(): mozgatja a buszt a tervezett következő mező felé (Vehicle-ből felüldefiniálva)
- void collideWith(Vehicle v): ütközést kezel egy másik járművel, beállítja a mozgásképtelen állapotot
- void slip(): jégpáncélon történő megcsúszást kezel

3.3.3 Car (Autó)

Felelősség

A városlakók automatizált járműve, amely önállóan navigál lakása és munkahelye között a legrövidebb járható útvonalon. Ha vékony hórétegen halad, letapossa a havat; jégen csúszhat és balesetet okozhat; mély hóban elakad, de szomszédos szabad sávba átsorolhat.

Ősosztályok

Vehicle

Interfészek

Nincs

Attribútumok

- Building homeBuilding: a lakás épülete, ahonnan indul
- Building workBuilding: a munkahely épülete, ahova tart
- Map gameMap: a térkép referenciája az útvonal-számításhoz

Metódusok

- void move(): a kocsi a kiszámolt útvonal alapján lép a következő mezőre (Vehicle-ből felüldefiniálva)
- void calculatePath(): meghívja a gameMap.findShortestPath()-t és beállítja a nextField-et
- void collideWith(Vehicle v): ütközés kezelése, sávlezárás meghatározott körre
- void slip(): jégpáncélon megcsúszás kezelése
- void changeLane(): átsorolás a szomszédos szabad sávba elakadás esetén

3.3.4 CleanerHead <<abstract>> (Hókotró fej)

Felelősség

A hókotróra szerelhető cserélhető eszközök absztrakt ősosztálya. Meghatározza, hogy a hókotró hogyan hat a sávon lévő hóra és jégre. Minden konkrét fejnek implementálnia kell a clean(Lane) metódust.

Ősosztályok

Nincs

Interfészek

IPurchasable

Attribútumok

- int price: a fej ára a boltban
- bool usesFuel: használ-e üzemanyagot

Metódusok

- void clean(Lane lane): a konkrét fejre jellemző takarítási műveletet elvégzi az adott sávon (absztrakt)
- int getPrice(): visszaadja a fej árát (IPurchasable-ből)
- String getName(): visszaadja a fej nevét (IPurchasable-ből)

3.3.5 DragonHead (Sárkány fej)

Felelősség

A legdrágább és leghatékonyabb hókotró fej. Gázturbinával azonnal elolvasztja a sávon lévő teljes havat és jégpáncélt. Működéséhez biokerozin szükséges; ha a készlet kiürül, a clean() nem hat.

Ősosztályok

CleanerHead

Interfészek

Nincs

Attribútumok

- double fuelAmount: a jelenleg tárolt biokerozin mennyisége
- double fuelPrice: a tárolt üzemanyag ára

Metódusok

- void clean(Lane lane): azonnal eltávolítja az összes havat és jeget a sávról, felhasználva a biokerozinból a szükséges mennyiséget (CleanerHead-ből felüldefiniálva)
- void refillFuel(double amount): biokerozint tölt a tartályba a telephelyen

3.3.6 Field <<interface>> (Mező)

Felelősség

A gráf alapköve: minden pályaelem (Lane, Cross, Building leszármazottjai) ezt az interfészt valósítja meg. Biztosítja, hogy a járművek egységesen tudjanak mozogni a gráfban, és hogy minden csomópont ismerje szomszédait.

Ősosztályok

Nincs

Attribútumok

–

Metódusok

- void accept(Vehicle v): fogadja az érkező járművet erre a mezőre
- void remove(Vehicle v): eltávolítja a távozó járművet erről a mezőről
- List<Field> getNeighbors(): visszaadja a szomszédos mezők listáját

3.3.7 GameLogic (Játéklogika)

Felelősség

A játék menetét vezérlő osztály. Felelőssége: a körök szervezése (tervezési fázis → végrehajtási fázis), az autók navigálásának indítása, a játék végének detektálása és az eredmény összesítése. A havazás szimulálása a Map.snow() metóduson keresztül történik.

Ősosztályok

Nincs

Interfészek

Nincs

Attribútumok

- Map gameMap: a játék térképe
- List<Player> players: a játékban részt vevő játékosok listája
- int turnCount: az aktuális kör sorszáma
- int maxTurns: a játék hossza körökben
- List<Vehicle> vehicles: az összes jármű listája

Metódusok

- void startGame(): inicializálja és elindítja a játékot
- void advanceTurn(): eggyel növeli a körszámlálót, végrehajtja a végrehajtási fázist
- void manageTurn(): kezeli az aktív játékosok döntési fázisát
- void endGame(): befejezi a játékot és kiszámítja az eredményt

3.3.8 GeneralBuilding (Lakás/Munkahely)

Felelősség

Az autók által felkeresett általános épület (lakás vagy munkahely). A Building absztrakt osztályból származik, tehát maga is Field a gráfban. Felelőssége parkolóhelyet biztosítani az autóknak és navigációs célpontként szolgálni.

Ősosztályok

Building

Attribútumok

–

Metódusok

- void accept(Vehicle v): fogadja az érkező autót (Building-ből felüldefiniálva)
- void remove(Vehicle v): eltávolítja a távozó autót (Building-ből felüldefiniálva)
- List<Field> getNeighbors(): visszaadja a szomszédos mezőket (Field-ből)

3.3.9 HomeBase (Telephely)

Felelősség

A hókotrók bázisa, amely maga is Field a gráfban (Building leszármazott). Felelőssége a hókotrók fogadása, a fejcserék és az üzemanyag-utántöltések koordinálása. A beépített áruháznak is helyet ad.

Ősosztályok

Building

Attribútumok

- IntegratedMarket market: a telephelyen lévő bolt
- int capacity: a telephely maximális befogadóképessége

Metódusok

- void acceptSnowPlow(SnowPlow sp): fogadja a visszatérő hókotrót, belső kapacitáskezeléssel
- void refuelSnowPlow(SnowPlow sp, Fuel fuel): utántölti a megadott hókotró üzemanyag-tartályát
- void swapHead(SnowPlow sp, CleanerHead newHead): fejcserét hajt végre a megadott hókotróra
- void accept(Vehicle v): fogadja az érkező járművet (Building-ből felüldefiniálva)
- void remove(Vehicle v): eltávolítja a távozó járművet (Building-ből felüldefiniálva)
- List<Field> getNeighbors(): visszaadja a szomszédos mezőket (Field-ből)

3.3.10 IcebreakerHead (Jégtörő fej)

Felelősség

Speciális hókotró fej, amely a jégpáncél feltörésére specializálódott. A feltört jeget hó formájában az úton hagyja, a csúszásveszély megszűnik, de a sáv havassá válik. Havat nem takarít el, üzemanyagot nem igényel.

Ősosztályok

CleanerHead

Interfészek

Nincs

Attribútumok

–

Metódusok

- void clean(Lane lane): a sávon lévő jégpáncélt feltöri és havává alakítja, a csúszásveszélyt megszünteti (CleanerHead-ből felüldefiniálva)

3.3.11 IntegratedMarket (Beépített áruház)

Felelősség

A telephelyen lévő bolt, amelyből a takarítók eszközöket és üzemanyagot vásárolhatnak. Felelőssége az eladható termékek nyilvántartása és a vásárlási tranzakciók lebonyolítása.

Ősosztályok

Nincs

Interfészek

Nincs

Attribútumok

- List<IPurchasable> availableItems: a boltban kapható termékek listája

Metódusok

- boolean buyItem(Cleaner buyer, IPurchasable item): megvásárol egy terméket ha van elég pénz; igazzal tér vissza ha sikeres
- List<IPurchasable> getAvailableItems(): visszaadja a megvásárolható termékek listáját

3.3.12 IPurchasable <<interface>> (Vásárolható)

Felelősség

A boltban megvásárolható termékeket összefogó interfész. Minden vásárolható terméknek (hókotró fej, üzemanyag, hókotró) meg kell valósítania.

Ősosztályok

Nincs

Attribútumok

Nincs

Metódusok

- int getPrice(): visszaadja a termék árát
- String getName(): visszaadja a termék nevét

3.3.13 Lane (Sáv)

Felelősség

Az út egyetlen forgalmi sávja és egyben Field a gráfban. Felelőssége nyilvántartani a hóréteg vastagságát, a jégpáncél állapotát és a rajta tartózkodó járműveket. Kezeli a letaposás miatti jégképződési mechanizmust.

Ősosztályok

Nincs

Interfészek

Field

Attribútumok

- double snowDepth: a sávon lévő hóréteg vastagsága
- bool isFrozen: igaz, ha a sávon jégpáncél van
- double iceThickness: a jégpáncél vastagsága
- int compactionCount: hányszor hajtottak át járművek – küszöb felett jégpáncél képződik
- List<Vehicle> vehicles: a sávon tartózkodó járművek listája
- double saltEffect: a sóhatás fennmaradásának ideje
- List<Field> neighbors: a szomszédos mezők (Field interfészből)

Metódusok

- void accept(Vehicle v): fogadja az érkező járművet (Field-ből felüldefiniálva)
- void remove(Vehicle v): eltávolítja a távozó járművet (Field-ből felüldefiniálva)
- List<Field> getNeighbors(): visszaadja a szomszédos mezőket (Field-ből)
- void addSnow(double amount): növeli a hóréteg vastagságát
- void removeSnow(double amount): csökkenti a hóréteg vastagságát
- void breakIce(): a jégpáncélt feltöri és havává alakítja
- void removeIce(): eltávolítja a jégpáncélt
- void checkFreeze(): ellenőrzi, hogy kell-e jégpáncél képződjön
- void applyCompaction(): növeli a compactionCount-ot járműáthaladáskor
- void applySaltEffect(double duration): beállítja a sóhatást a sávon
- boolean isBlocked(): igaz, ha a sáv járhatatlan

3.3.14 Map (Térkép)

Felelősség

Összefogja és kezeli az úthálózat összes csomópontját (Field-eket). Gráfként kezeli a pályát, és biztosít útvonal-kereső funkcionalitást. A havazást is ő koordinálja a snow() metóduson keresztül.

Ősosztályok

Nincs

Interfészek

Nincs

Attribútumok

- List<Field> fields: a térkép összes mezője (Lane, Cross, Building leszármazottjai)

Metódusok

- List<Field> findShortestPath(Field from, Field to): visszaadja a legrövidebb járható útvonalat
- void snow(): minden Lane típusú mezőn növeli a hóréteg vastagságát

3.3.15 Player <<abstract>> (Játékos)

Felelősség

A játékos absztrakt osztály, a Cleaner és a BusDriver közös ősosztálya. Tárolja az alapvető játékos-tulajdonságokat és előírja a körkezdeményező/-záró metódusokat.

Ősosztályok

Nincs

Interfészek

Nincs

Attribútumok

- String name: a játékos neve
- int score: a játékos aktuális pontszáma

Metódusok

- void startTurn(): elkezdi az adott játékos körét (absztrakt)
- void endTurn(): befejezi az adott játékos körét (absztrakt)

3.3.16 Road (Út)

Felelősség

Az úthálózat egy útját reprezentálja. Felelőssége az utak végpontjainak és szomszédainak számontartása, valamint összefogja a sávokat, és épületeket.

Ősosztályok

Nincs

Interfészek
Nincs

Attribútumok
- List<Road> startPoint: a szomszédos utak listája
- List<Road> endPoint: a szomszédos utak listája
- List<Lane> forwardLanes: a startból endbe haladó sávok listája
- List<Lane> backwardLanes: az endből startba haladó sávok listája
- List<Building> buildings: az utcához csatlakozó épületek
- Double length: az út hossza

Metódusok
- List<Road> getStartConnections(): visszaadja az út elején a szomszédos utak listáját
- List<Road> getEndConnections(): visszaadja az út végén a szomszédos utak listáját
- List<Lane> getForwardLanes(): Visszaadja a forwardLane-eket
- List<Lane> getBackwardLanes(): Visszaadja a BackwardLane-eket

3.3.17 SaltHead (Sószóró fej)

Felelősség
Sót szóró hókotró fej. A kiszórt só idővel felolvasztja a havat és jégpáncélt, és megakadályozza az újabb hó megmaradását. Működéséhez só szükséges; kiürüléskor hatástalan.

Ősosztályok
CleanerHead

Interfészek
Nincs

Attribútumok
- double fuelAmount: a jelenleg tárolt só mennyisége
- double fuelPrice: a tárolt üzemanyag ára

Metódusok
- void clean(Lane lane): sót szór a sávra, beállítja a sóhatás időtartamát; ha nincs só, nem csinál semmit (CleanerHead-ből felüldefiniálva)
- void refillFuel(double amount): sót tölt a tartályba a telephelyen

3.3.18 SnowPlow (Hókotró)

Felelősség
A takarító játékos által irányított munkagép. A rászerelt fej segítségével takarítja a sávokat, pénzt termel a takarítónak. Hóláncai miatt jégen sem csúszik. A telephelyen tud fejet cserélni és utántölteni.

Ősosztályok
Vehicle

Interfészek
IPurchasable

Attribútumok
- List<CleanerHead> attachments: a jelenleg birtokolt hókotró fejek listája
- CleanerHead activeHead: a jelenleg felszerelt hókotró fej

Metódusok
- void move(): mozgatja a hókotrót a tervezett útvonal szerint (Vehicle-ből felüldefiniálva)
- void changeHead(CleanerHead newHead): kicseréli az aktív fejet a megadottra (csak telephelyen)
- void cleanCurrentLane(): a jelenlegi sávon végrehajtja az aktív fejre jellemző takarítást
- int getPrice(): visszaadja a hókotró árát (IPurchasable-ből)
- String getName(): visszaadja a termék nevét (IPurchasable-ből)

3.3.19 SweepHead (Söprő fej)

Felelősség

A hókotróra szerelhető söprő típusú fej. A havat a szomszédos sávba vagy az út mellé tolja. Jeget nem tud eltávolítani. Üzemanyagot nem igényel.

Ősosztályok

CleanerHead

Interfészek

Nincs

Attribútumok

–

Metódusok
- void clean(Lane lane): a sáv havát áttol a szomszédos sávba vagy az út szélére; jégre nincs hatása (CleanerHead-ből felüldefiniálva)

3.3.20 Terminal (Végállomás)

Felelősség

A buszjáratok végpontja, egyben Field a gráfban (Building leszármazott). Felelőssége a fordulók regisztrálása: amikor egy busz megérkezik, jelzi a buszvezetőnek a forduló teljesítését.

Ősosztályok

Building

Attribútumok

–

Metódusok
- void registerArrival(Bus bus): regisztrálja a busz érkezését és jelzi a fordulót a buszvezetőnek
- void accept(Vehicle v): fogadja az érkező buszt (Building-ből felüldefiniálva)
- void remove(Vehicle v): eltávolítja a távozó buszt (Building-ből felüldefiniálva)
- List<Field> getNeighbors(): visszaadja a szomszédos mezőket (Field-ből)

3.3.21 ThrowerHead (Hányó fej)

Felelősség

A söprőhöz hasonló hókotró fej, de a havat jóval messzebbre (pl. a járdára) szórja, nem csupán a szomszédos sávba. Jeget nem tud eltávolítani. Üzemanyagot nem igényel.

Ősosztályok

CleanerHead

Interfészek

Nincs

Attribútumok

- double throwDistance: az a távolság, ameddig a hányó fej a havat elhajítja

Metódusok

- void clean(Lane lane): a sáv havát messzire szórja el az útról; jégre nincs hatása (CleanerHead-ből felüldefiniálva)

3.3.22 Vehicle <<abstract>> (Jármű)

Felelősség

A játékban szereplő összes jármű absztrakt ősosztálya. Tárolja a jármű aktuális sávját és a tervezett következő mezőt, és előírja a move() metódust.

Ősosztályok

Nincs

Interfészek

Nincs

Attribútumok

- Lane currentLane: a jármű aktuális sávja
- Field nextField: a tervezett következő mező

Metódusok

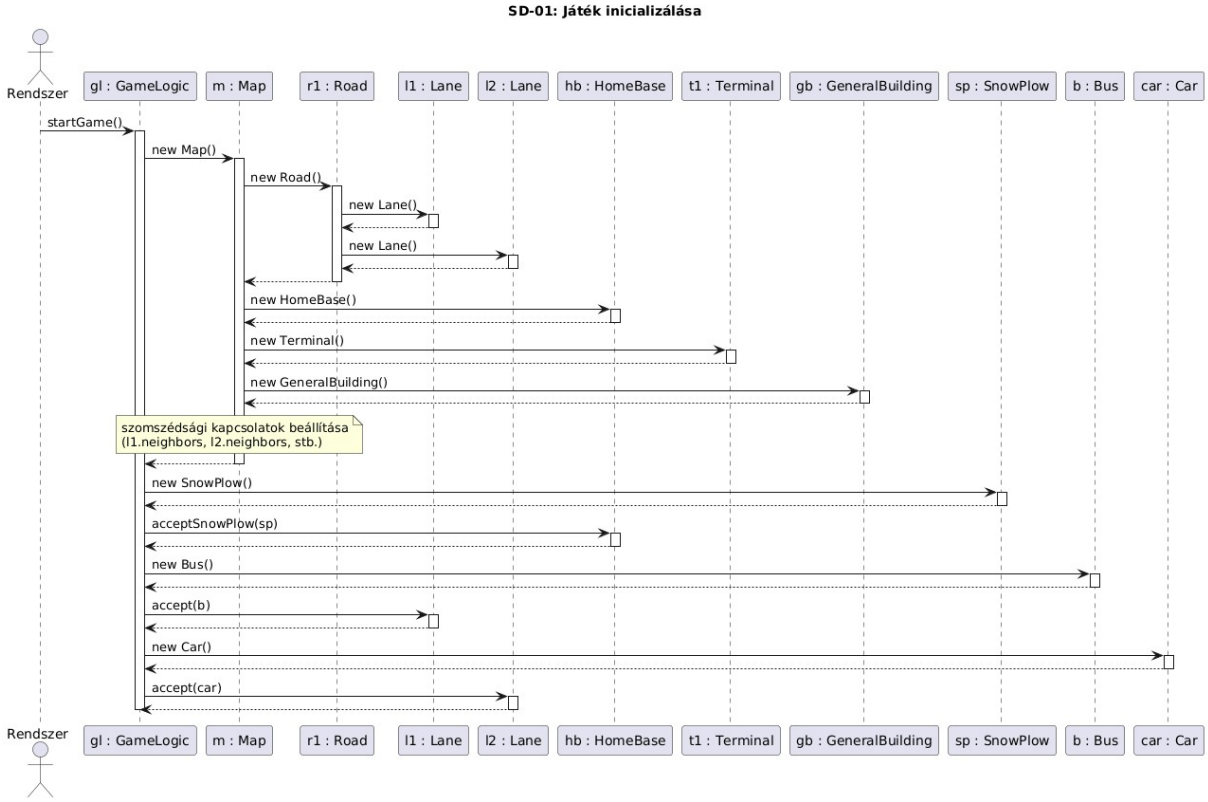
- void move(): a jármű mozgatása a jelenlegi helyzetről a következő tervezett mezőre (absztrakt)

3.4 Szekvencia diagramok

Az alábbi szekvenciadiagramok a játék legfontosabb use-case-eit és belső működési eseményeit mutatják be. Minden diagram egyetlen belépési ponttal rendelkezik, a metódushívások egymásba ágyazottan követik a hívási vermet.

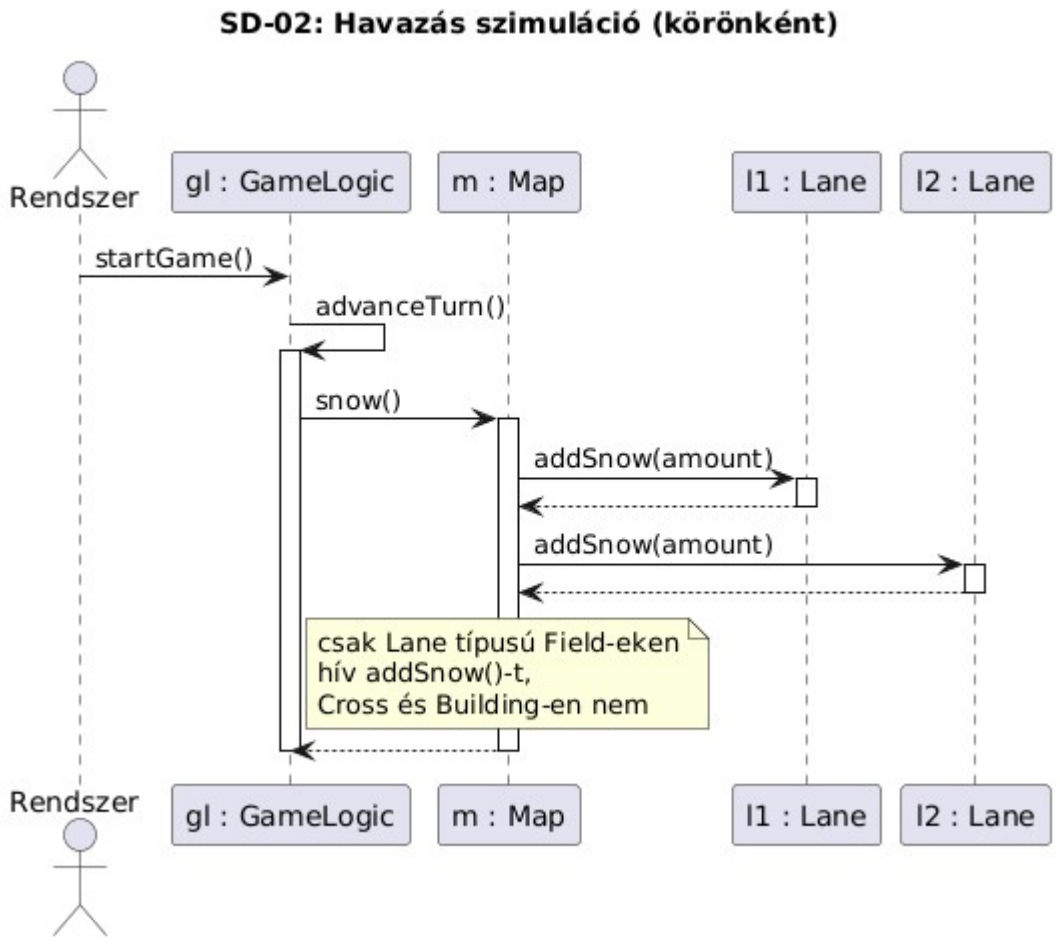
SD-01: Játék inicializálása

A GameLogic létrehozza a Map-et, feltölti azt Field objektumokkal (Lane, Building leszármazottjai). A mezők szomszédsági kapcsolatait beállítja (getNeighbors-on keresztül elérhető gráf). Ezután létrehozza a játékosokat, a járműveket, és elhelyezi őket a megfelelő Lane-ekre az accept(v) hívással.



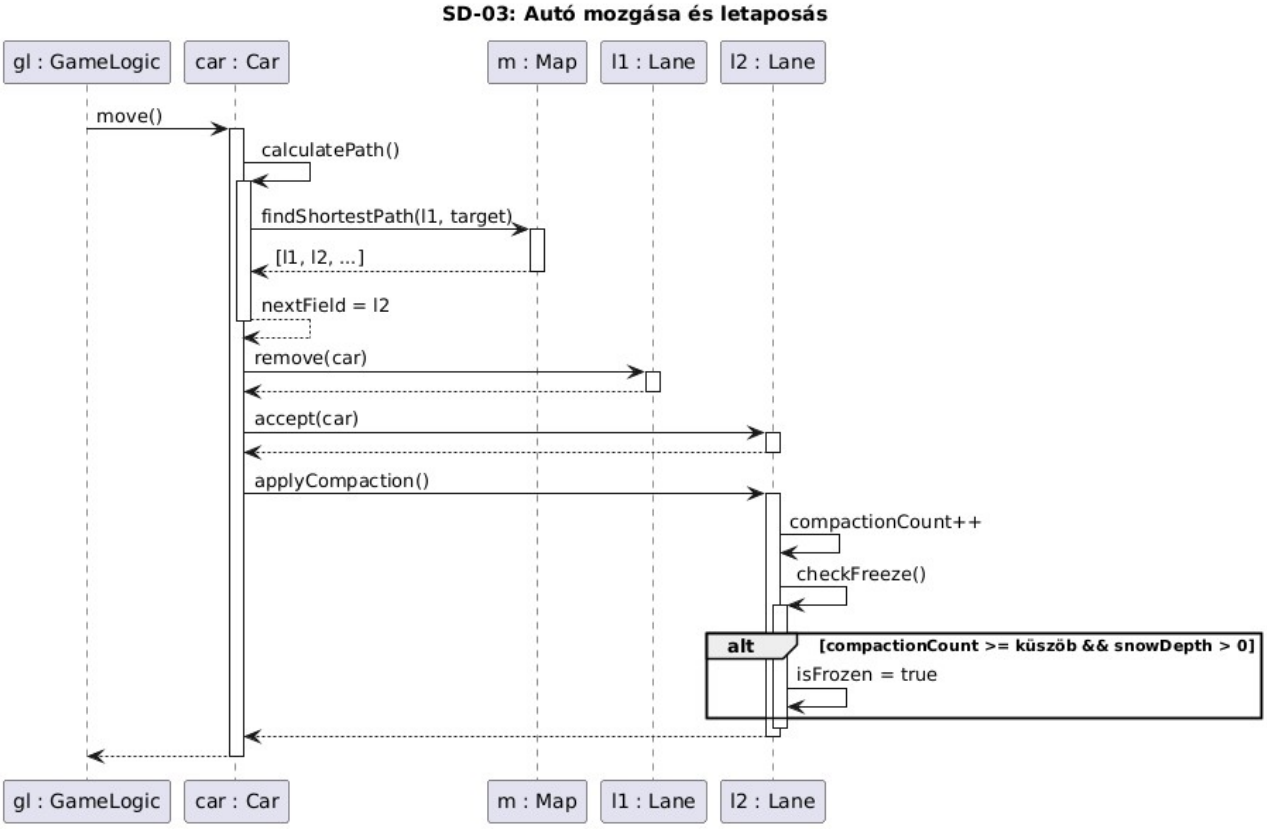
SD-02: Havazás szimuláció (körönként)

A GameLogic.advanceTurn() meghívja a Map.snow() metódust. A Map.snow() végigiterál a fields listán, és minden Lane típusú mezőn meghívja az addSnow(amount) metódust. Más Field típusokon (Building) nem hív addSnow()-t.



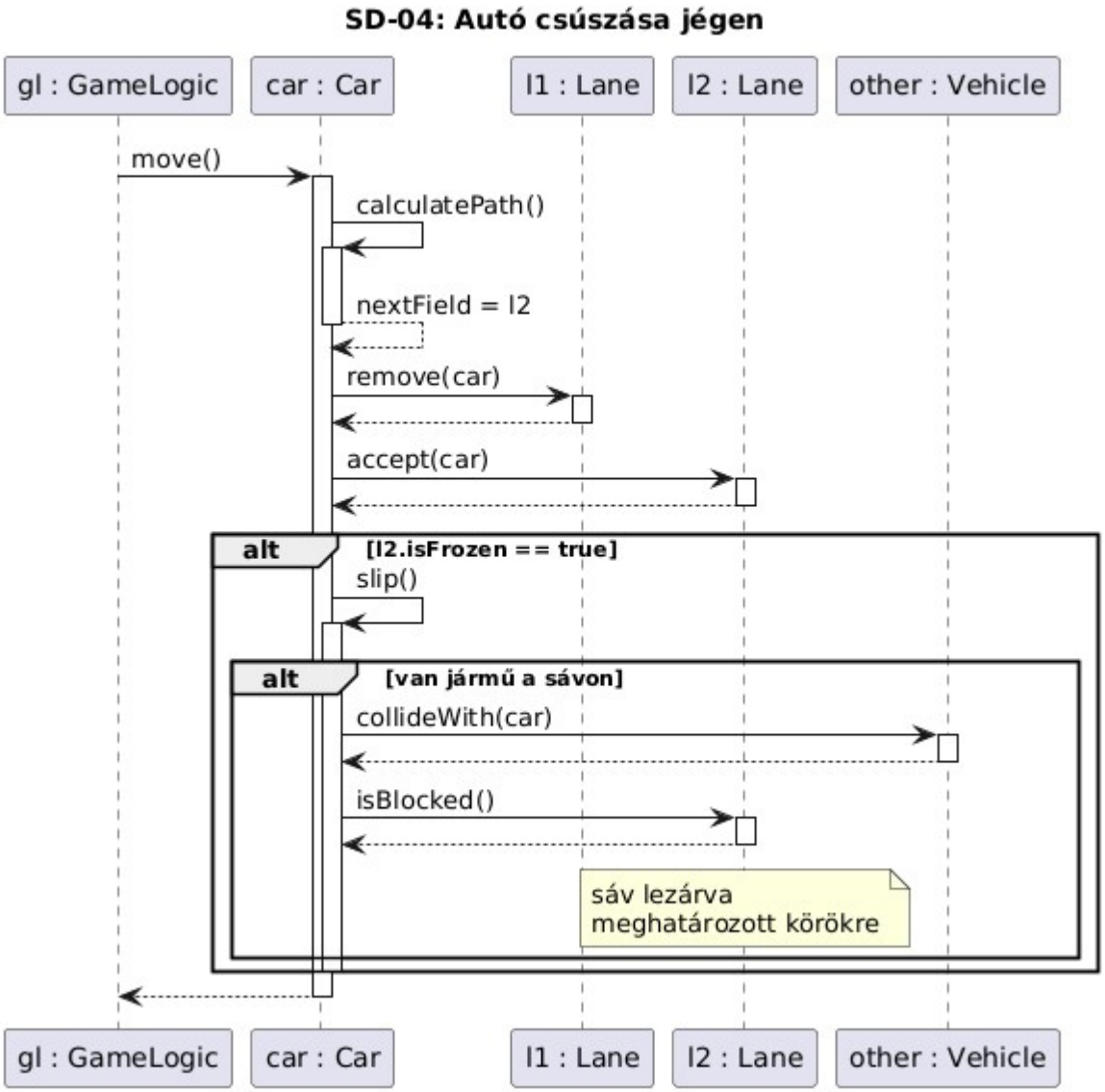
SD-03: Autó mozgása és letaposás

A GameLogic.advanceTurn() meghívja az összes Car.move() metódusát. A Car.move() meghívja a Car.calculatePath()-t, amely meghívja a gameMap.findShortestPath(currentLane, target)-t. A következő Field accept(car) hívódik, az előző remove(car) hívódik. Ha a célmező Lane, meghívja a lane.applyCompaction()-t, majd a lane.checkFreeze()-t.



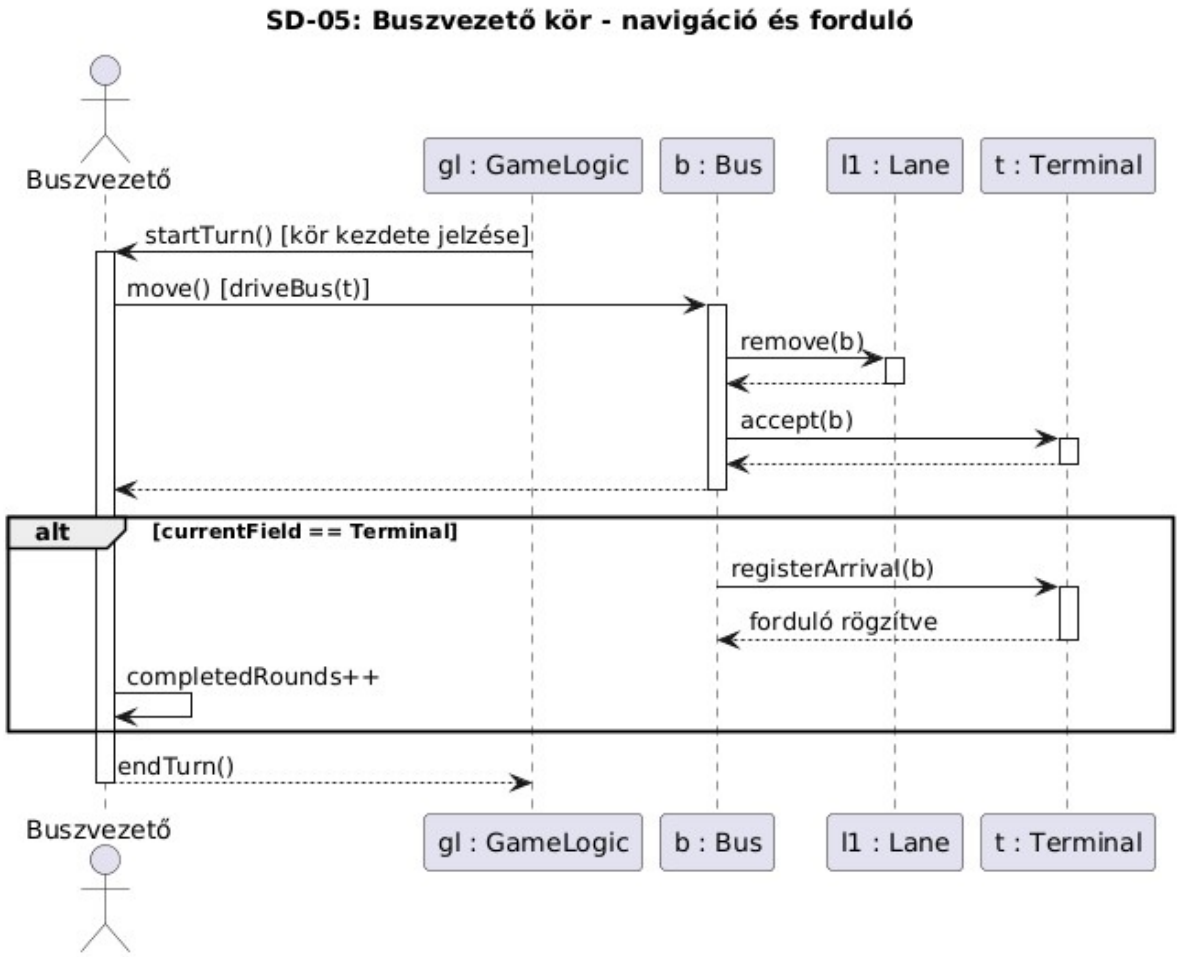
SD-04: Autó csúszása jégen

Egy Car.move() során a célLane.isFrozen igaz. A Lane jelzi a csúszásveszélyt, véletlenszerű eséllyel a Car.slip() meghívódik. A Car.slip() meghívja a collideWith(Vehicle) metódust a közelben lévő járművön (ha van), az érintett Lane.isBlocked() igazra áll.



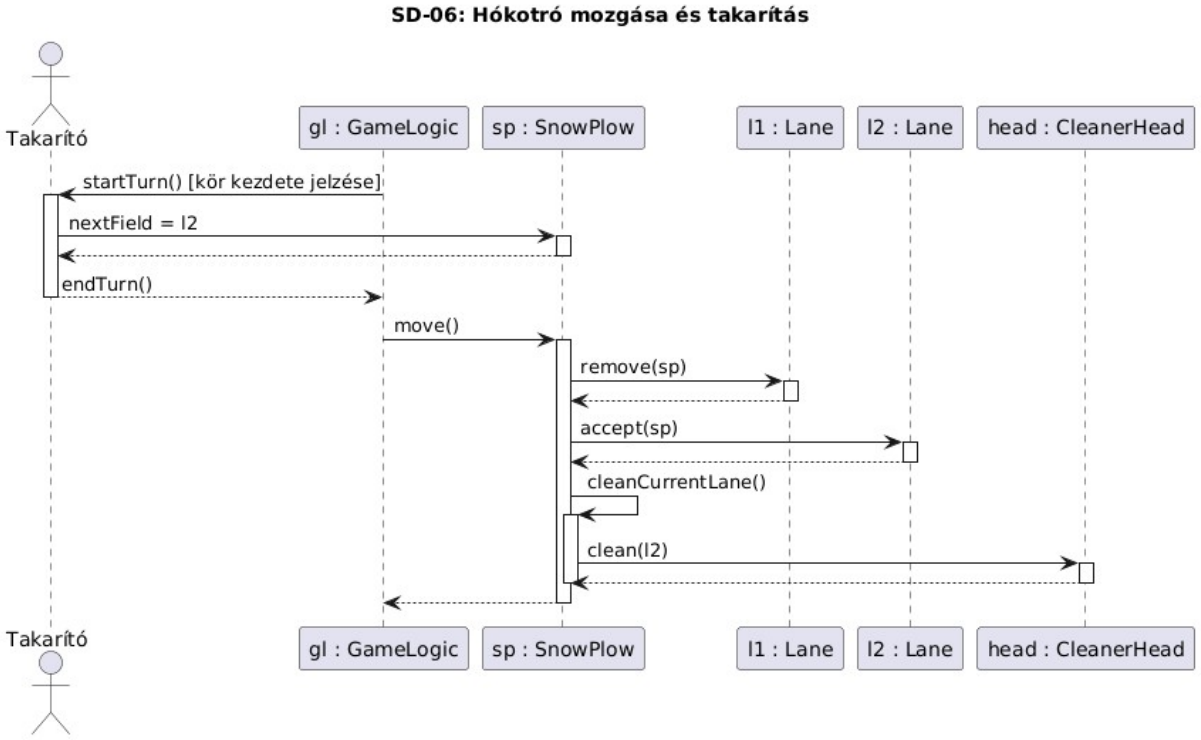
SD-05: Buszvezető kör – navigáció és forduló

A GameLogic.manageTurn() jelzi a buszvezető aktornak a kör kezdetét. A buszvezető kiválasztja a következő célmezőt, a Bus.move() a célmező accept(bus)-t és az előző remove(bus)-t hívja. A Bus.move() végén ellenőrzi, hogy Terminal-ra érkezett-e; ha igen, a Terminal.registerArrival(bus) regisztrálja a fordulót.



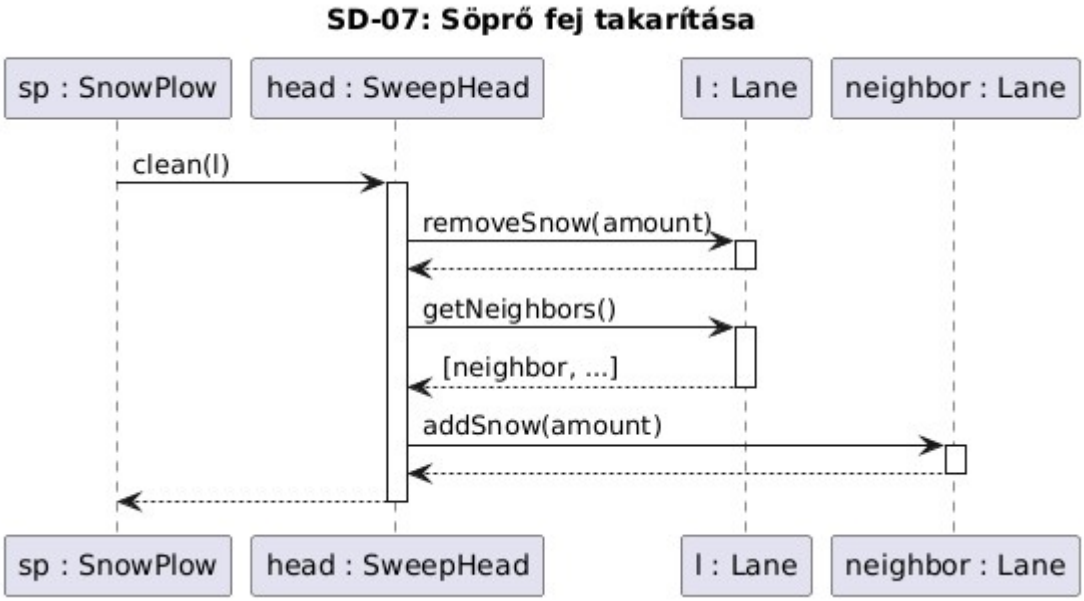
SD-06: Hókotró mozgása és takarítás

A takarító aktor beállítja a hókotró nextField-jét. A GameLogic.advanceTurn() meghívja a SnowPlow.move()-t: a célmező accept(snowplow), az előző remove(snowplow). Ezután a SnowPlow.cleanCurrentLane() meghívja az activeHead.clean(currentLane) metódust.



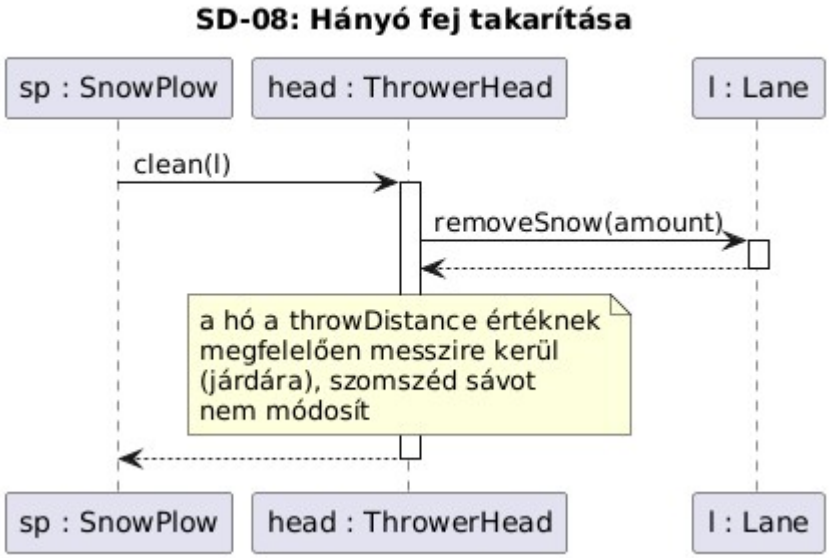
SD-07: Söprő fej takarítása

A SweepHead.clean(lane) meghívja a lane.removeSnow(amount)-t, majd a lane.getNeighbors()-ból megkapja a szomszéd Lane-t, végül a neighborLane.addSnow(amount)-t hívja – áttolva a havat.



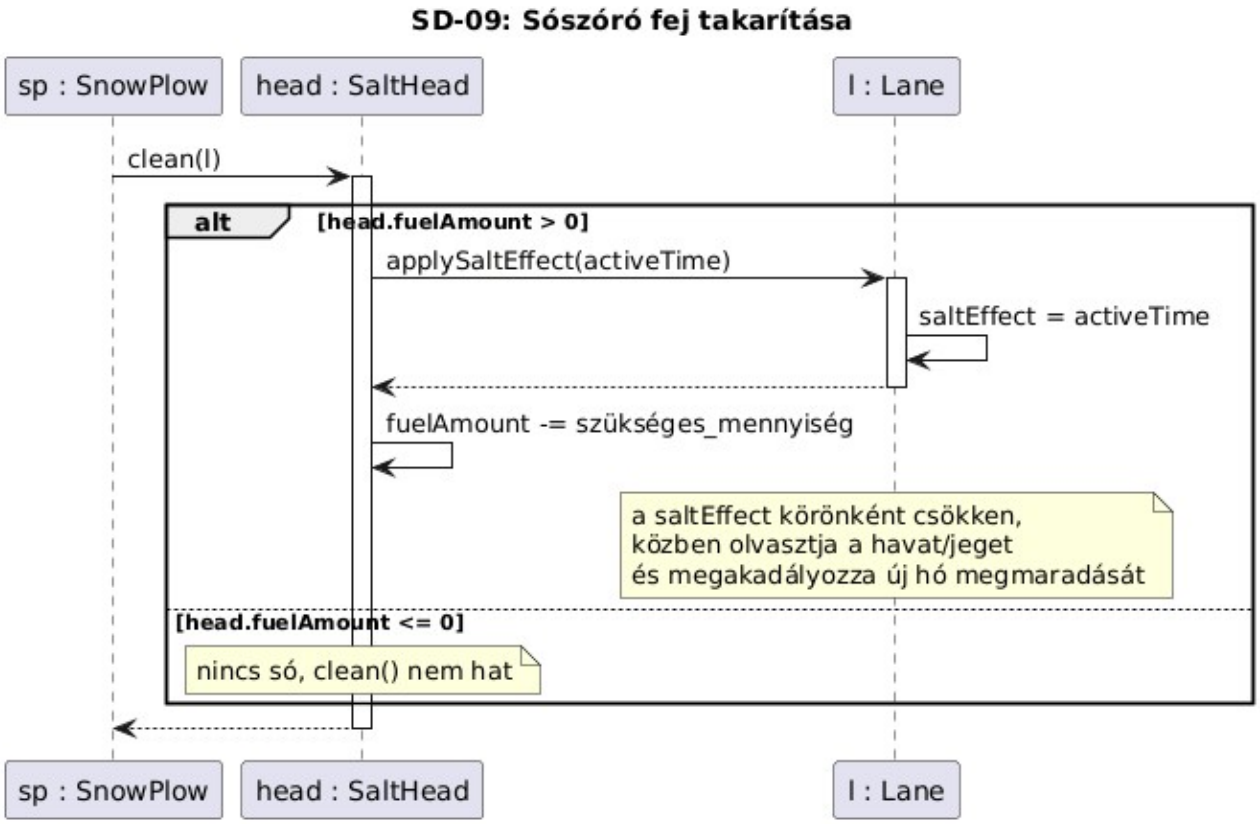
SD-08: Hányó fej takarítása

A ThrowerHead.clean(lane) meghívja a lane.removeSnow(amount)-t. A throwDistance értékének megfelelően a hó messzire kerül – nem a szomszéd sávba, hanem az út szélére (nem kell szomszédos Lane-t módosítani).



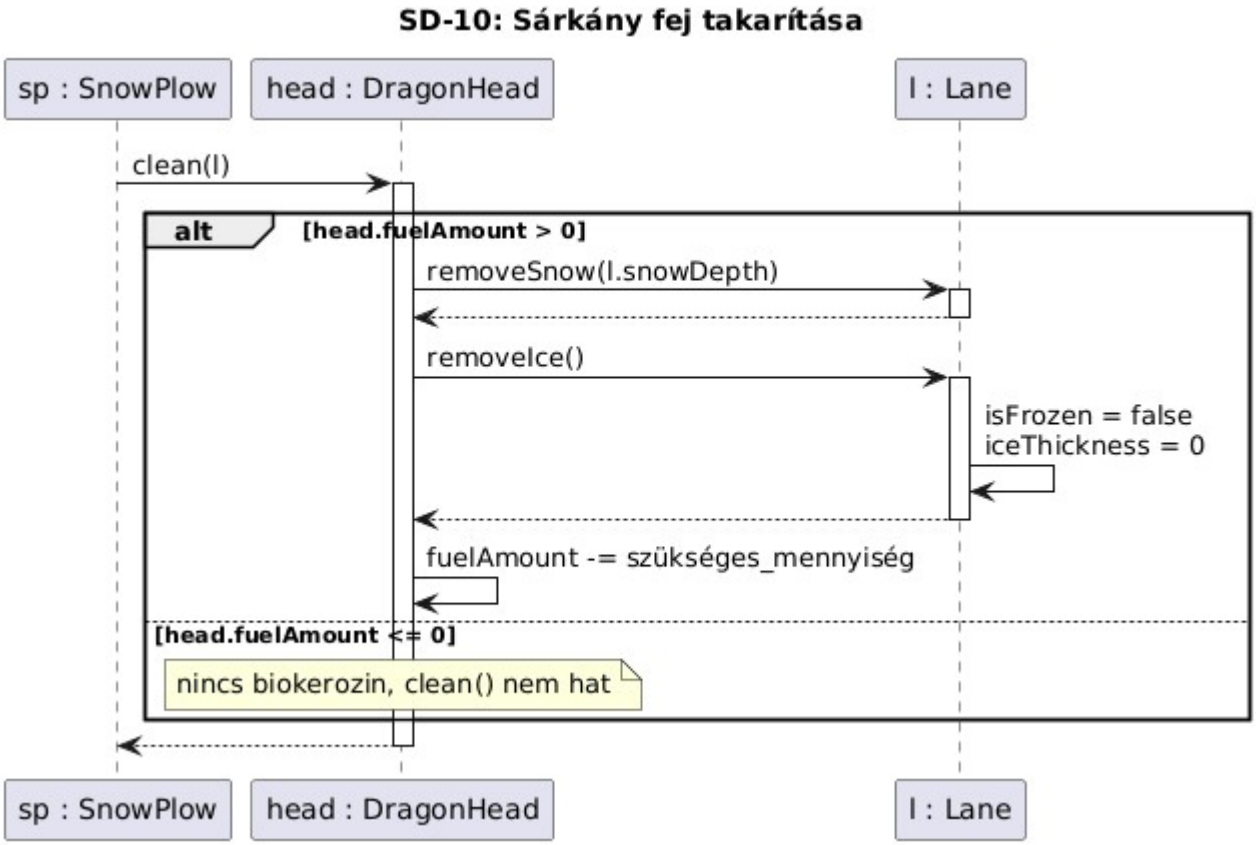
SD-09: Sószóró fej takarítása

A SaltHead.clean(lane) ellenőrzi, hogy fuelAmount > 0. Ha igen, meghívja a lane.applySaltEffect(activeTime)-t és csökkenti a fuelAmount-ot. A Lane.applySaltEffect() beállítja a saltEffect értékét, amely körönként csökkentve fokozatosan olvasztja a havat és jeget.



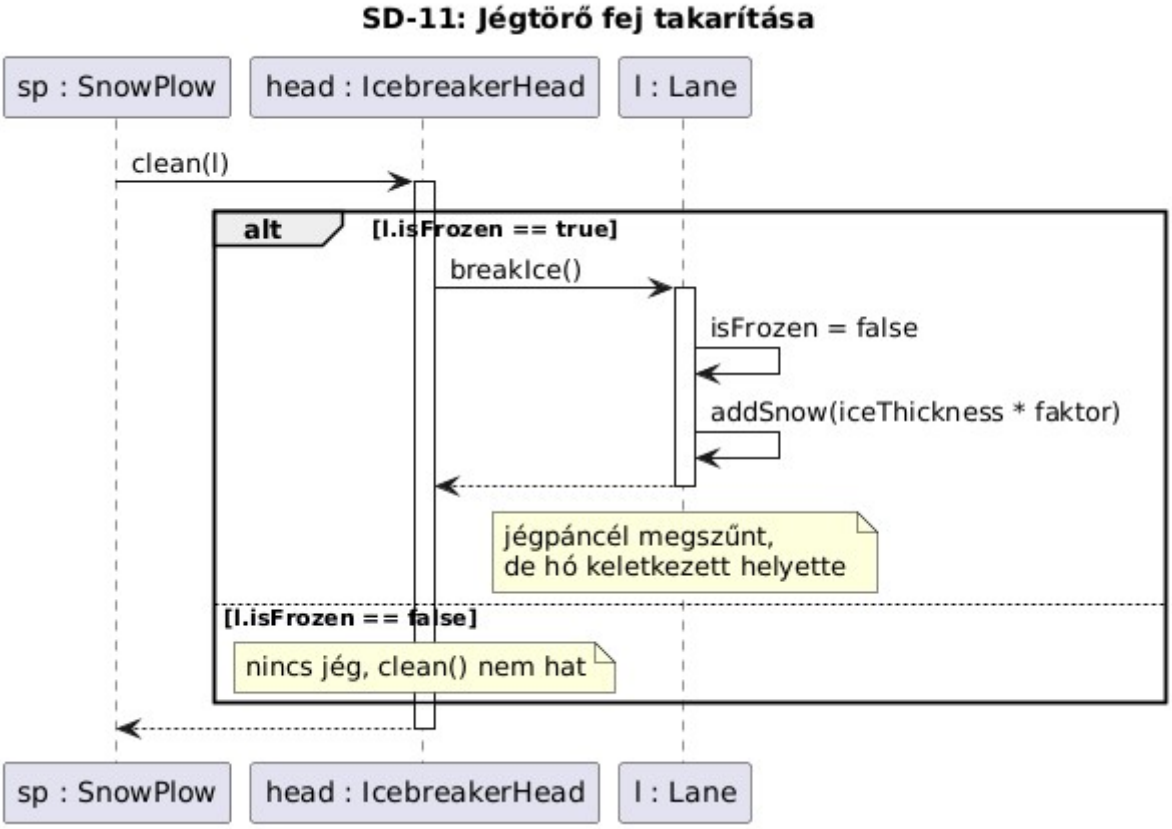
SD-10: Sárkány fej takarítása

A DragonHead.clean(lane) ellenőrzi, hogy fuelAmount > 0. Ha igen, meghívja a lane.removeSnow(lane.snowDepth)-t, majd a lane.removeIce()-t, és csökkenti a fuelAmount-ot.



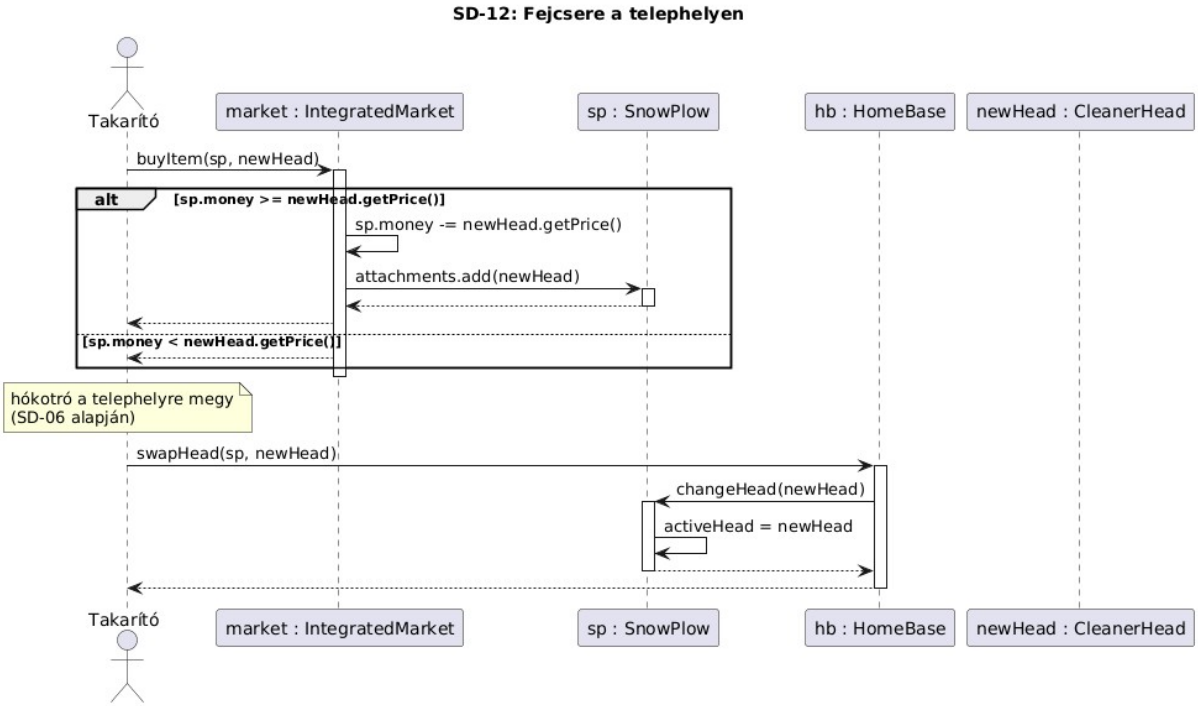
SD-11: Jégtörő fej takarítása

Az IcebreakerHead.clean(lane) meghívja a lane.breakIce()-t, amely beállítja az isFrozen=false-t és a jégvastagságot hóvá alakítja (lane.addSnow(iceThickness * faktor)).



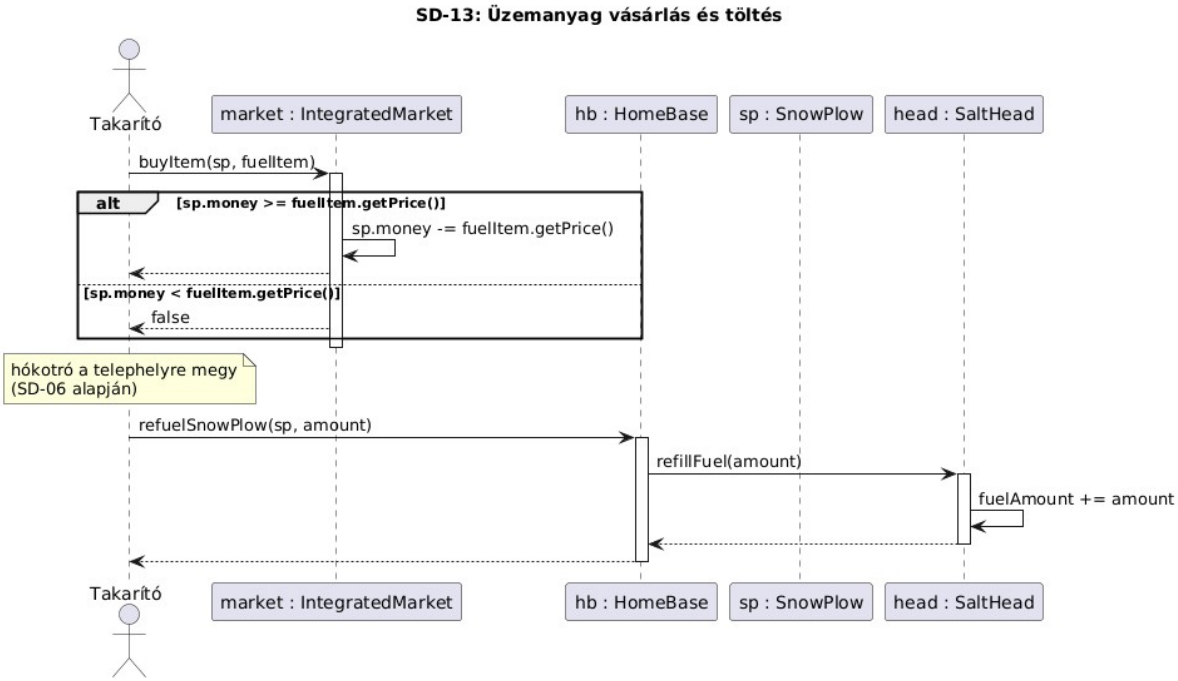
SD-12: Fejcsere a telephelyen

A takarító aktor kezdeményezi a vásárlást. Az IntegratedMarket.buyItem(sp, head) sikeres tranzakció esetén a fejet hozzáadja a hókotró attachments listájához. Ezután HomeBase.swapHead(sp, newHead) meghívja a SnowPlow.changeHead(newHead)-et, beállítva az activeHead-et.



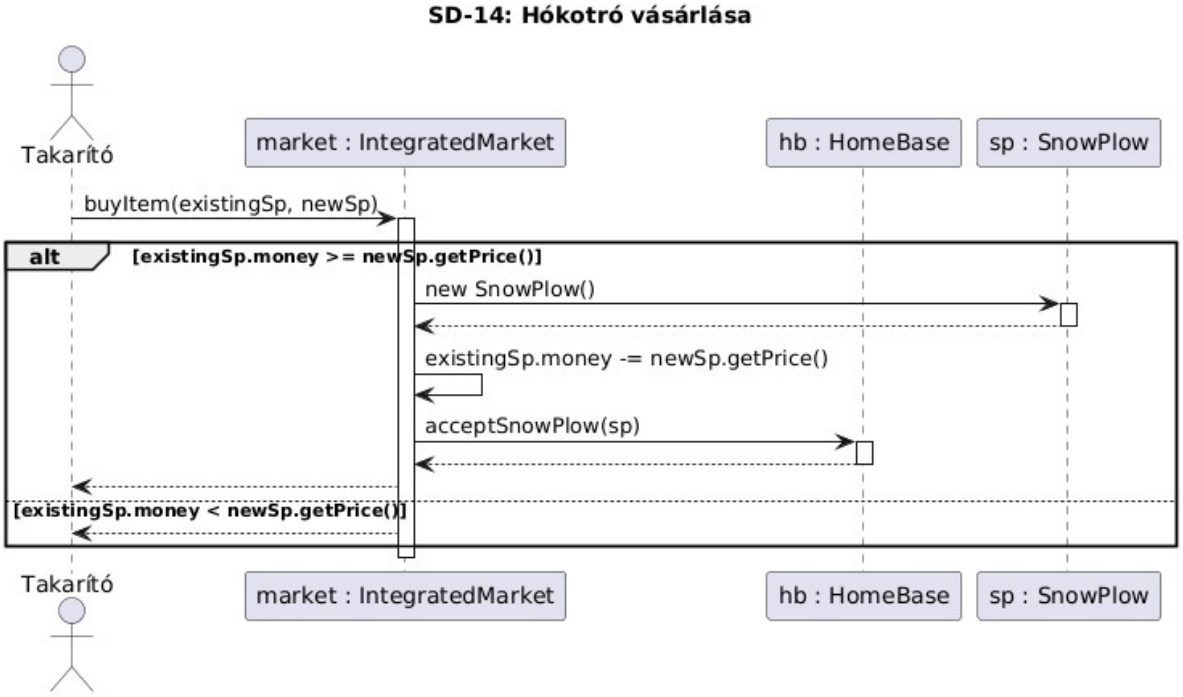
SD-13: Üzemanyag vásárlás és töltés

A takarító aktor kezdeményezi az üzemanyag vásárlást az IntegratedMarket-en. Ezután HomeBase.refuelSnowPlow(sp, amount) meghívja a SaltHead.refillFuel(amount) vagy DragonHead.refillFuel(amount) metódust.



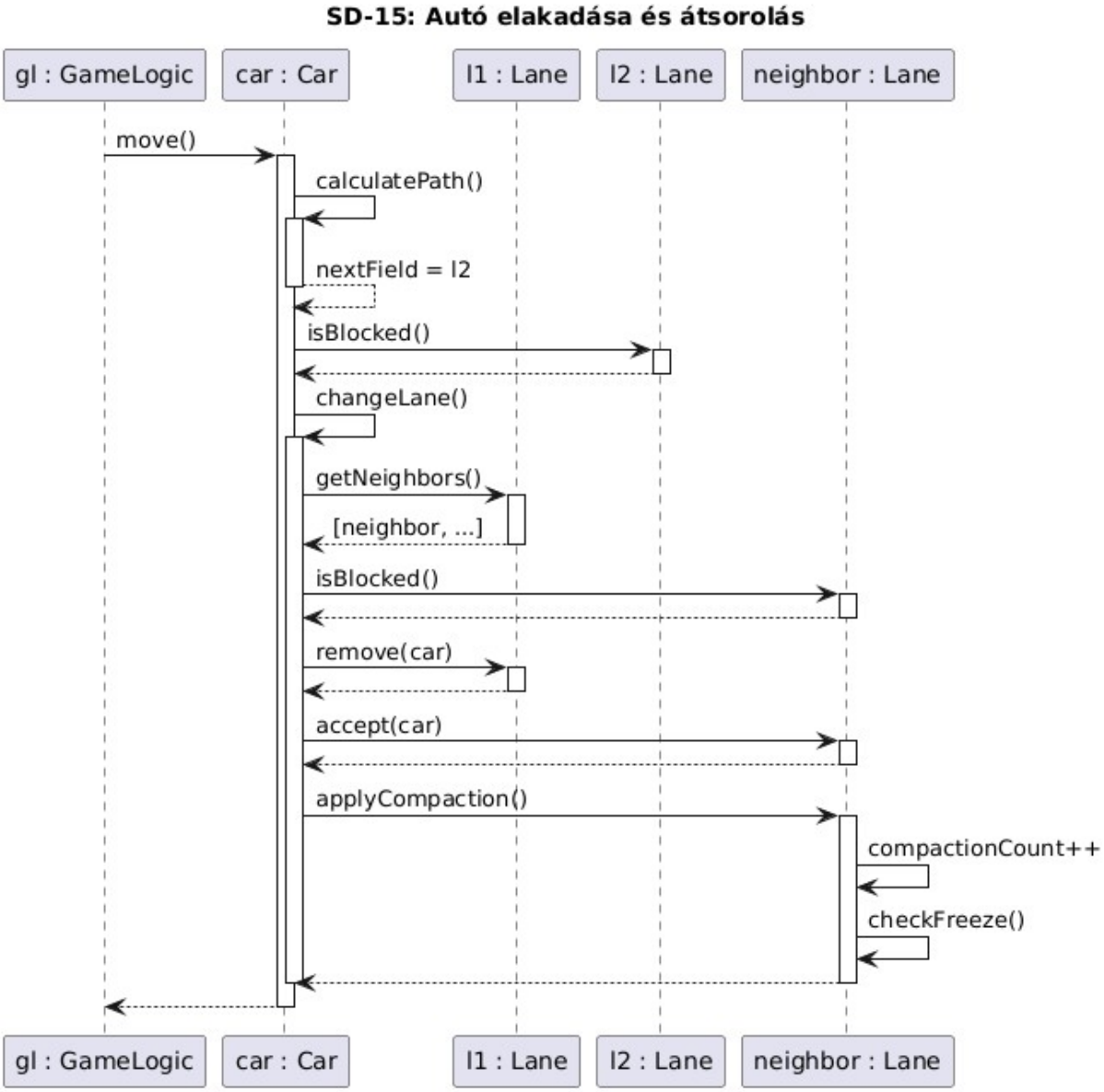
SD-14: Hókotró vásárlása

A takarító aktor kezdeményezi a hókotró vásárlást. Az IntegratedMarket.buyItem() sikeres tranzakciója után egy új SnowPlow példány jön létre. A HomeBase.acceptSnowPlow(sp) fogadja az új hókotrót..



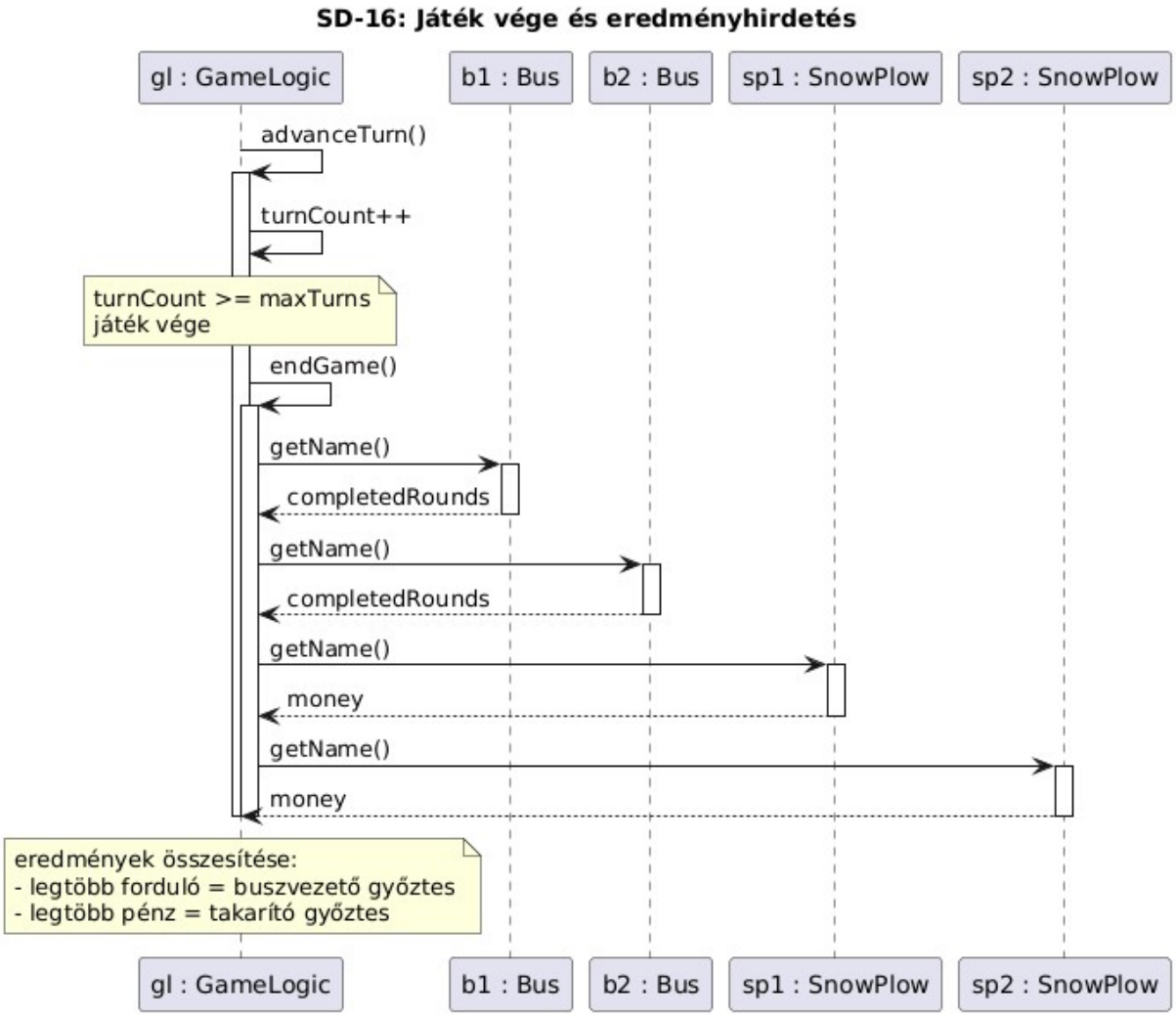
SD-15: Autó elakadása és átsorolás

A GameLogic.advanceTurn() észleli, hogy turnCount >= maxTurns. Meghívja a GameLogic.endGame()-t, amely minden Bus esetén összesíti a completedRounds-ot, minden SnowPlow esetén összesíti a pénzt, majd az eredményt visszaadja.



SD-16: Játék vége és eredményhirdetés

A GameLogic.advanceTurn() észleli, hogy turnCount >= maxTurns. Meghívja a GameLogic.endGame()-t, amely minden BusDriver esetén összesíti a completedRounds-ot, minden Cleaner esetén összesíti a money-t, majd az eredményt visszaadja.

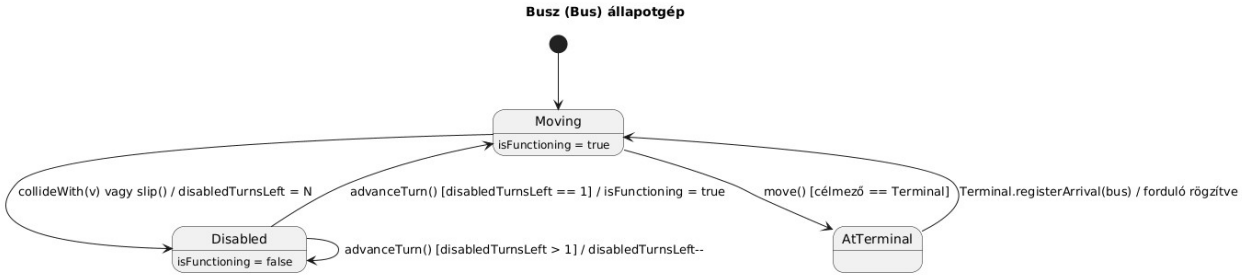


3.5 State-chartok

Az alábbi állapotgépek azokhoz az osztályokhoz készültek, amelyek viselkedése állapotfüggő és több mint egy állapottal rendelkeznek.

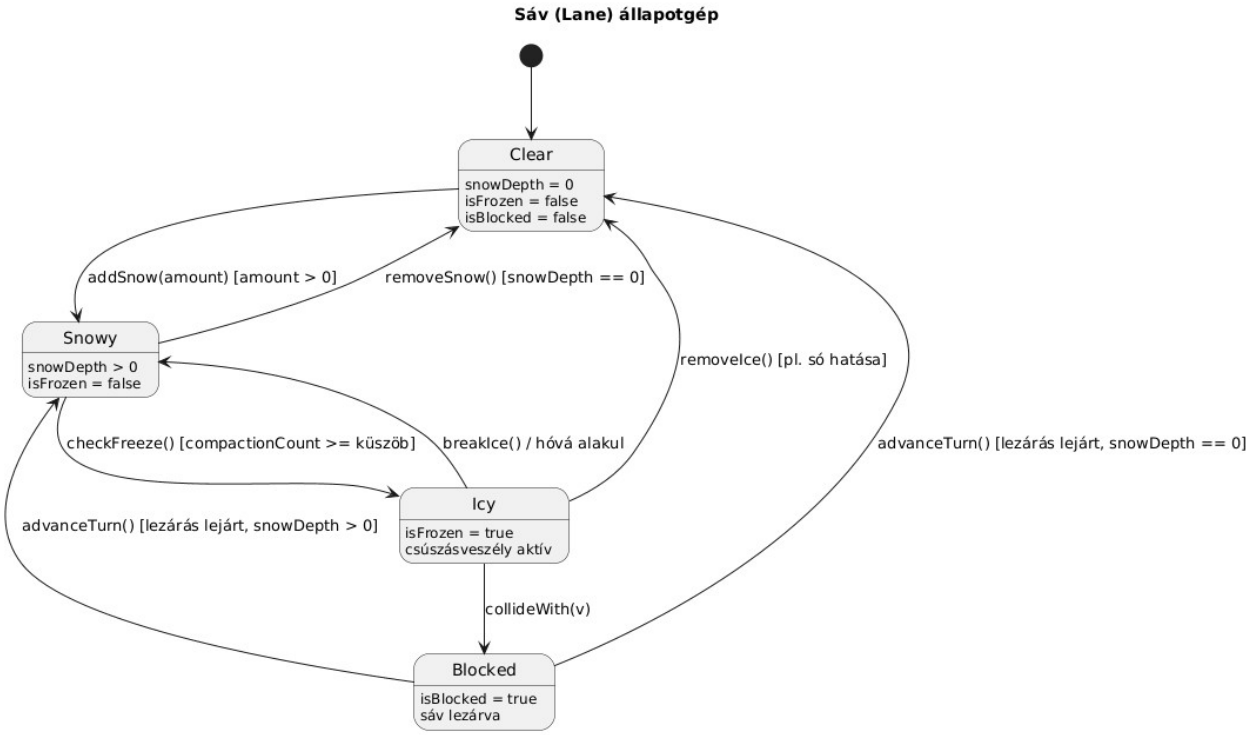
Busz (Bus) állapotgép

A busz három állapotban lehet. Mozgás (Moving): normálisan közlekedik a buszvezető irányításával. Mozgásképtelen (Disabled): ütközés vagy csúszás után; a disabledTurnsLeft csökkentésével tér vissza Mozgásba. Végállomáson (AtTerminal): megérkezett a végállomásra, a Terminal.registerArrival() hívódik, majd visszamegy Mozgásba.



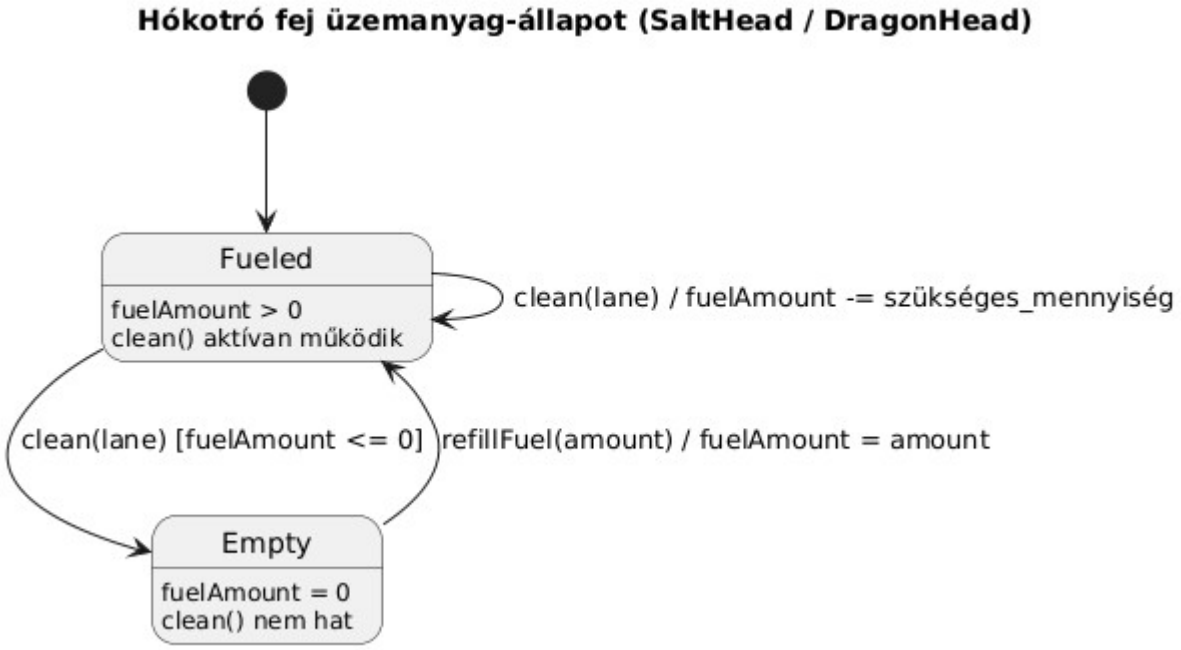
Sáv (Lane) állapotgép

A sáv négy állapotban lehet. Tiszta (Clear): nincs hó, nincs jég; havazás hatására átmegy Havas állapotba. Havas (Snowy): hóréteg van rajta; elég letaposásra Jeges állapotba megy, takarítással visszamegy Tiszta állapotba. Jeges (Icy): jégpáncél van rajta; csúszásveszélyes; IcebreakerHead vagy SaltHead hatására visszamegy Havas vagy Tiszta állapotba. Lezárt (Blocked): ütközés miatt; meghatározott körök után visszanyílik.



Hókotró fej üzemanyag-állapot (SaltHead / DragonHead)

Két állapot: Üzemanyaggal (Fueled): a fej aktívan működik, minden clean() hívás csökkenti a fuelAmount-ot. Üres (Empty): fuelAmount = 0; a clean() nem csinál semmit. A telephelyen történő refillFuel() visszaviszi Fueled állapotba.



Napló

Kezdet	Időtartam	Résztvevők	Leírás
2026.03.05:00	1 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: Megállapodtunk mit javítottunk ki..
2026.03.06 13:00	2 óra	Kálmán	Tevékenység: A diagrammokat javitotta.
2026.03.06. 18:00	2 óra	Héja	Tevékenység: Javította az osztályok leírásának nagy részét.
2026.03.07. 02:00	2 óra	Papp	Tevékenység: Javította a szekvencia diagrammokat.
2026.03.08. 10:00	2 óra	Grób	Tevékenység: egységesítette az osztályokat, és a leírásukat. Ellenőrizte a konzisztenciát
2026.03.08. 14:00	2 óra	Dobronyi	Tevékenység: Egységesítette a dokumentumot, ellenőrizte, és hibajavította.
2026.03.08. 18:00	3 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: összeegyeztettük az egyénileg megcsinált részfeladatokat, és véglegesítettük a dokumentumot.